

Backups

Allgemeines

Was ist eigentlich ein Backup?

Übersetzt man die Begrifflichkeit von der englischen in die deutsche Sprache, dann ist ein Backup nichts anderes als eine Sicherung. Im Rahmen der Informatik spricht man hierbei von der Sicherung von Daten. Daten können dabei Dinge sein wie:

- Dateien
- Verzeichnisse
- Datenbanken
- Komplette Partitionen
- Virtuelle Rechner

Die Daten werden dabei auf entsprechende Speichermedien kopiert. Zu diesen Medien gehören beispielsweise externe Festplatten, optische Datenträger wie CDs oder DVDs, USB-Sticks oder auch entsprechende Speicherplätze innerhalb der Cloud. (vgl. [BAC1])

Warum werden Backups gemacht?

Daten sind ein sehr vulnerables Gut. Daten können versehentlich gelöscht werden. Daten können allerdings auch verloren gehen, da zum Beispiel die Festplatte oder der ganze Rechner auf dem sie gespeichert sind kaputt gehen. Auch das Einschleusen von Schadsoftware kann dazu führen, dass Daten nicht mehr genutzt werden können. Die Schadsoftware "Emotet" zum Beispiel verschlüsselt Daten und verlangt für die Entschlüsselung eine Lösegeldzahlung. Hierbei handelt es sich nur um einige wenige Beispiele, die aufzeigen, wie wichtig es ist, dass Daten regelmäßig gesichert werden und somit diese auch wiederhergestellt werden können! (vgl. [BAC1] [EMO1])

Archivierung != Backup

Häufig sprechen Menschen bei einem Backup auch von Archivierung. Zwischen den beiden Begrifflichkeiten herrscht jedoch ein großer Unterschied. Bei der Archivierung ist die Rede von **Revisions- bzw. Beweissicherheit**. Daten müssen besonders wegen rechtlichen Gründen langfristig aufbewahrt werden. Wichtig hierbei ist, dass die Daten nicht verändert werden dürfen. Das heißt, dass z. B. Unternehmen auch nachweisen müssen, dass nach einer Datenspeicherung die Daten auch nicht mehr manipuliert wurden. Daten werden folglich auf ein entsprechendes Speichermedium verschoben und nach dem **WORM** Prinzip abgelegt. WORM steht hierbei für **Write Once Read Many**. Daten werden somit einmalig abgelegt und können im Anschluss nur noch gelesen, aber nicht mehr verändert oder gelöscht werden. Hierfür eignen sich vor allem optische Speichermedien. (vgl. [BAC2])

Vergleicht man folglich ein Backup mit einer Archivierung, so wird man feststellen, dass ein Backup hauptsächlich durch regelmäßige Sicherungskopien versucht, die Gefahr vor Datenverlust zu minimieren. Archivierungen hingegen stellen die Unveränderlichkeit und Nachvollziehbarkeit von Daten für rechtliche und regulatorische Zwecke sicher. (vgl. [BAC2])

Wohin werden Daten gesichert?

Es gibt generell die unterschiedlichsten Möglichkeiten, wohin Daten gesichert werden können. Die gängigsten Optionen sind dabei:

Bandsicherungen

Bandsicherungen werden auch als **Tape Backups** bezeichnet. Hierbei werden Daten auf Magnetbändern gespeichert. Diese Art der Sicherung wird vor allem in Unternehmen genutzt, da sie sehr kostengünstig ist und eine hohe Speicherkapazität bietet. (vgl. [BAC3])

Dateisysteme

Daten können auch auf Dateisystemen gesichert werden. Hierbei werden Daten auf Festplatten oder SSDs gespeichert. Aber auch Lösungen wie NAS und SAN sind hierbei zu nennen.

Festplatten / SSDs

Bei HDDs, also Hard Disk Drives, handelt es sich um eine Technologie, bei der bewegliche, rotierende magnetische Platten zur Datenspeicherung eingesetzt werden. SSDs, also Solid State Drives, hingegen speichern Daten auf Flash-Speicherchips. Im Vergleich zur Geschwindigkeit sind SSDs schneller als HDDs, da keine mechanischen Komponenten bewegt werden müssen. Auch in Bezug auf die Langlebigkeit liegen SSDs vor den HDDs, da es keine mechanischen Bestandteile gibt, die verschleifen könnten. SSDs sind allerdings noch immer teurer als HDDs. (vgl. [BAC4])

NAS

Der **Network Attached Storage** oder kurz **NAS** ist ein Speichermedium, das über das Netzwerk angesprochen wird und somit einer Gruppe von Benutzern innerhalb des Netzwerks einen Datenzugriff bieten kann. Ein NAS System ist in der Regel mit mehreren Festplatten ausgestattet wodurch eine erhöhte Ausfallsicherheit ermöglicht wird und gleichzeitig eine gute Speicherkapazität geboten werden kann. NAS Systeme sind kostengünstige und auch leicht zu wartende Netzwerkspeicheroptionen. (vgl. [BAC5])

SAN

Das **Storage Area Network** oder kurz **SAN** ist ein Netzwerk, das speziell für die Speicherung von Daten konzipiert ist. SANs sind in der Regel sehr leistungsstark und bieten eine hohe Verfügbarkeit. SANs sind allerdings auch sehr teuer und benötigen spezielle Hardware. Für die Benutzer im Netzwerk wirkt es, als wäre jeder User mit dem entsprechenden Speicherpool direkt verbunden. (vgl. [BAC5])

Optische Datenträger

Optische Datenträger sind Speichermedien, die hauptsächlich für die Archivierung von Daten genutzt werden. Hierzu zählen CDs, DVDs und Blu-Rays. Diese Medien sind sehr kostengünstig und bieten eine hohe Langlebigkeit. Besonders für die Archivierung werden gerne optische Datenträger genutzt, da hier das WORM Prinzip angewendet werden kann. Die Datenträger werden einmalig beschrieben und können später in der Regel nicht mehr verändert werden. Die Datenträger selbst sind auch sehr widerstandsfähig gegenüber Umwelteinflüssen, bieten eine kostengünstige Möglichkeit zur langfristigen Speicherung an, aber weisen meist geringe Speicherkapazitäten auf. (vgl. [BAC6])

Off-Premise Speicher

Bei dieser Art der Speicherlösung werden die Daten an einem externen Ort außerhalb des eigenen Unternehmens bzw. des eigenen Rechenzentrums abgelegt. Zusätzlich zum Aspekt der allgemeinen Datensicherheit wird hierbei noch Sicherheit und Schutz vor lokalen Katastrophen wie Einbruch, Feuer oder Überschwemmung geboten. Es gibt mehrere unterschiedliche Möglichkeiten, was ein Off-Premise Speicher sein kann: (vgl. [BAC7])

Externe Rechenzentren / Datacenter

Die eigenen Daten werden in einem Rechenzentrum abgelegt, welches von Drittanbietern gespeichert wird. Ein typisches Beispiel könnte sein, dass zwei Niederlassungen gegenseitig ihre Daten im jeweils anderen Rechenzentrum der anderen Niederlassung speichern. (vgl. [BAC7])

Cloud Speicher

Anbieter wie Amazon Web Services, Microsoft Azure oder Google Cloud bieten die Möglichkeit, dass sämtliche Daten innerhalb der Cloud gespeichert werden. Dabei werden klar die Daten innerhalb der Rechenzentren abgelegt, die von den jeweiligen Anbietern betrieben werden. Der Vorteil ist natürlich die hohe Verfügbarkeit und die entsprechende leichte Skalierbarkeit und Verwaltung. Allerdings liegen die Daten bei externen Anbietern, was für viele Unternehmen ein Problem darstellt. (vgl. [BAC7])

Offsite Backups

Offsite Anbieter bieten die Möglichkeit, dass Daten innerhalb externer Rechenzentren und zusätzlich auf externen Speichermedien gesichert werden. Mehr oder weniger können somit zwei Sicherungen von Daten vorgenommen werden. Die physischen Medien wie Bänder oder Festplatten werden an einem anderen Ort gelagert z. B. in einem Tresor, einem Schließfach oder bei einem Archivierungsdienst. Dadurch wird eine zusätzliche Sicherheit vor lokalen Katastrophen geboten. (vgl. [BAC7])

Sicherungsarten

Images und Abbilder

Ein Image bzw. ein Abbild ist mehr oder weniger eine Kopie eines Datenträgers bzw. eines ganzen Systems. Es kann sich dabei um eine Kopie einer Festplatte, eines USB-Sticks oder einer CD/DVD handeln. Ein Image kann auch eine Kopie eines Betriebssystems oder einer virtuellen Maschine sein. (vgl. [IMG1])

Blockbasierte Images

Ein blockbasiertes Image ist eine Kopie eines Datenträgers, die auf Blockebene erstellt wird. Dabei wird der Datenträger in Blöcke unterteilt und diese Blöcke werden dann kopiert. Ein blockbasiertes Image enthält alle Daten, die sich auf dem Datenträger befinden, einschließlich des Dateisystems und sämtlicher Dateien. Ein blockbasiertes Image kann auf einem anderen Datenträger wiederhergestellt werden, um den ursprünglichen Datenträger zu ersetzen. (vgl. [IMG2])

Sicherungen unter Linux

Unter Linux gibt es das Kommando `dd`. `dd` steht dabei für `Disk Dump` und ist ein Programm, wodurch blockbasierte Images erstellt und somit Sicherungen durchgeführt werden können. Die wesentliche Syntax von `dd` ist:

```
dd if=<Source> of=<Destination>
```

TERMINAL

Mit dem Parameter `if`, welcher für `Input File` steht, wird die Quelle angegeben, die gesichert werden soll. `of` für `Output File` hingegen spezifiziert das Ziel für die eigentliche Sicherung. `dd` kann somit auch verwendet werden um Dateien einfach nur zu kopieren:

```
dd if=/root/example.txt of=/tmp/example.txt
```

TERMINAL

Möchte man eine komplette Partition sichern, so kann ebenfalls der `dd` Befehl genutzt werden:

```
dd if=/dev/sda1 of=/tmp/dump.img
```

TERMINAL

Um letztendlich das Image einer z. B. Partition wieder einspielen zu können, wird der Befehl einfach umgedreht:

```
dd if=/tmp/dump.img of=/dev/sda1
```

TERMINAL

Sicherungen einer virtuellen Maschine

Unter Windows können zum Beispiel auch die virtuellen Maschinen auf einer Hyper-V gesichert werden. Hierzu existiert das Kommando `get-vm` und `export-vm`:

```
get-vm | export-vm -path d:\backup\
```

TERMINAL

Das Kommando `get-vm` listet dabei alle vorhandenen, virtuellen Maschinen auf der Hyper-V auf. `export-vm` mit dem zusätzlichen Parameter `-path` ist in der Lage, dass für jede virtuelle Maschine eine eigene Datei im angegebenen Pfad erzeugt wird.

Um letztendlich die virtuellen Maschinen wieder herstellen zu können, gibt es das Kommando `import-vm`:

```
import-vm d:\backup\<Name_of_VM>.vmcx
```

TERMINAL

IMPORTANT

Das Exportieren sowie das Importieren von virtuellen Maschinen als auch das Sichern und Wiederherstellen von Images kann grundlegend im laufenden Betrieb erfolgen. Es besteht hierbei allerdings immer das Risiko, dass Inkonsistenzen aufgrund von geöffneten Dateien etc. entstehen. Daher empfiehlt es sich, dass Sicherungen standardmäßig zu Zeiten durchgeführt werden, an denen relativ wenige Betrieb ist z. B. in der Nacht!.

Dateisicherungen

Dateisicherungen repräsentieren die Sicherung von kompletten Verzeichnissen und deren beinhalteten Dateien. Im Gegensatz zu Images bzw. Abbildern, welche komplette Datenträger wie CDs/DVDs, USB-Sticks oder Festplatten sichern, wird bei einer Dateisicherung explizit ausgewählt, was in die Sicherung einfließen soll und was nicht. (vgl. [SIC1])

Arten von Datensicherungen

Vollsicherung

Wie der Name vermuten lässt, wird bei einer Vollsicherung der komplette Datenbestand, wie er zum Zeitpunkt der Sicherung vorliegt, gesichert. Dies bedeutet, dass alle Dateien und Verzeichnisse in die Sicherung einfließen. Zu den Vorteilen der Vollsicherung zählen, dass immer der komplette Datenbestand gesichert wird. Man kann sich somit auch sicher sein, dass alles, was benötigt wird, wirklich vorhanden ist. (vgl. [SIC2])

Inkrementelle Sicherung

Im Gegensatz zu einer Vollsicherung wird bei einer inkrementellen Sicherung nur der Datenbestand gesichert, der sich seit der letzten Sicherung geändert hat. Dies bedeutet, dass nur die Dateien und Verzeichnisse gesichert werden, die seit der letzten Sicherung neu hinzugekommen oder geändert wurden. Um dieses Vorhaben zu verwirklichen, wird der Zeitstempel der Dateien und Verzeichnisse mit dem Zeitstempel der letzten Sicherung verglichen. Aufgrund dieser Tatsache ist eine inkrementelle Sicherung wesentlich schneller als eine Vollsicherung und benötigt weiterhin weniger Speicherplatz. (vgl. [SIC2])

Differentielle Sicherung

Das differentielle Backup ähnelt der ersten durchgeführten inkrementellen Sicherung. Die Daten seit der letzten Vollsicherung werden gesichert. Dieses Prinzip wird allerdings bei jeder einzelnen differentiellen Sicherung erneut durchgeführt. Das bedeutet, dass bei einer differentiellen Sicherung immer der Datenbestand gesichert wird, der sich seit der letzten Vollsicherung geändert hat. (vgl. [SIC2])

Übersicht Backups

Nummer des Backups	Vollsicherung	Inkrementelles Backup	Diferenzielles Backup
1	Alle Daten	-	-
2	Alle Daten	Änderungen seit Backup Nr. 1	Änderungen seit Backup Nr. 1
3	Alle Daten	Änderungen seit Backup Nr. 2	Änderungen seit Backup Nr. 1
4	Alle Daten	Änderungen seit Backup Nr. 3	Änderungen seit Backup Nr. 1

Sicherungsstrategien

Sicherungsstrategien sind ein wichtiger Bestandteil der Datensicherung. Sie legen fest, wie und in welchem Zeitraum Sicherungen durchgeführt werden. Es gibt verschiedene Arten von Sicherungsstrategien, die sich in der Häufigkeit und dem Umfang der Sicherungen unterscheiden. Grundsätzlich sollte eine vollständige Sicherung allerdings mindestens einmal pro Woche stattfinden. Welche Sicherungen zwischen den Wochentagen durchgeführt werden, hängt natürlich auch von den Anforderungen des Unternehmens ab. Viele Unternehmen wählen aus der folgenden Liste eine Sicherungsstrategie aus: (vgl. [SIC2])

- Tägliche vollständige Sicherung
- Wöchentliche vollständige Sicherung + tägliche differenzielle Sicherung
- Wöchentliche vollständige Sicherung + tägliche inkrementelle Sicherung

Bei der Wahl der optimalen Sicherungsstrategie spielen viele Überlegungen eine Rolle. In der Regel muss bei jeder Alternative und Strategie ein Kompromiss zwischen Leistung, Data-Protection-Niveau, Gesamtmenge der aufbewahrten Daten und Kosten gefunden werden. Die Sicherungsstrategie hat Auswirkungen auf den Speicherplatz und resultiert in verschiedenen Anforderungen an die Medienkapazität und die für die Wiederherstellung erforderlichen Medien für die drei typische Sicherungsstrategien.

Generationenprinzip

Das Generationenprinzip wird auch sehr häufig als Großvater-Vater-Sohn-Prinzip beschrieben. Innerhalb der Datensicherung stellt das Prinzip sicher, dass immer mehrere Sicherungen in verschiedenen zeitlichen Abständen vorhanden sind, um somit jede mögliche Wiederherstellung verfügbar gemacht wird! Hauptsächlich bei der Sicherung auf Bändern wird dieses Prinzip sehr gerne verwendet, da hierbei auch sichergestellt wird, dass die Bänder nach einem bestimmten Schema bzw. einer bestimmten Zeitspanne wiederverwendet werden. (vgl. [GEN1])

Ein typisches Beispiel für das Generationenprinzip könnte die Sicherung in einer Fünf-Tage-Woche im Intervall eines Jahres darstellen. In einem solchen Intervall werden jeweils vier Sohn-Medien (= S1-S4) für die Tagessicherungen, vier Vater-Medien (= V1-V4) für die Wochensicherungen und zwölf Großvater-Medien (= G1-G12) für die Monatssicherungen verwendet. (vgl. [GEN1])

Betrachtet man zum Beispiel hierbei lediglich einen Monat und geht man der Einfachheit halber davon aus, dass die erste Woche im Monat mit einem Montag beginnt, dann sieht der Ablauf der Sicherung folgendermaßen aus:

1. Woche: Von Montag bis einschließlich Freitag werden jeweils Tagessicherungen, die sogenannten Sohn-Sicherungen, auf die Medien S1-S4 durchgeführt. Am Freitag wird die erste Wochensicherung, die Vater-Sicherung, auf das Medium V1 durchgeführt.

2. Woche: Das Prinzip von Woche 1 wiederholt sich hier. Von Montag bis einschließlich Donnerstag werden die Tagessicherungen auf die Medien S1-S4 durchgeführt. Am Freitag wird die zweite Wochensicherung auf das Medium V2 durchgeführt. Die Sicherungsmedien S1-S4 werden folglich immer wieder überschrieben.
3. und 4. Woche: Auch hier wiederholt sich das Prinzip von Woche 1 und Woche 2. Es kommt allerdings zum ersten Mal die Monatssicherung, die Großvater-Sicherung, auf das Medium G1 zum Einsatz. Die Sicherungsmedien S1-S4 und V1-V4 werden folglich immer wieder überschrieben.

Die 3-2-1 Regel

Bei der 3-2-1 Regel handelt es sich um eine bewährte Methode zur Sicherung von Daten, die darauf abzielt, das Risiko von Datenverlust zu minimieren. Jede Zahl innerhalb der Regel steht für eine spezifische Anforderung an die Datensicherung: (vgl. [BAC8])

- **3:** Es sollten mindestens drei Kopien der Daten vorhanden sein. Dies umfasst die Originaldaten und mindestens zwei Sicherungskopien. Dadurch wird sichergestellt, dass selbst im Falle eines Datenverlusts oder einer Beschädigung der Originaldaten immer noch zwei weitere Kopien zur Verfügung stehen. (vgl. [BAC8])
- **2:** Die Sicherungskopien sollten auf mindestens zwei verschiedenen Medientypen gespeichert werden. Dies kann beispielsweise eine Kombination aus Festplatten, optischen Medien und Cloud sein. Durch die Verwendung unterschiedlicher Medientypen wird das Risiko verringert, dass ein einzelner Medientyp ausfällt und alle Sicherungskopien verloren gehen. (vgl. [BAC8])
- **1:** Mindestens eine der Sicherungskopien sollte an einem externen Standort aufbewahrt werden, der sich physisch vom Standort der Originaldaten unterscheidet. Dies schützt die Daten vor lokalen Katastrophen wie Feuer, Überschwemmungen oder Diebstahl. (vgl. [BAC8])

Durchführung von Sicherungen via Skript

Der Befehl `tar`

`tar` ist die Abkürzung für `tape archive` und ein sehr weit verbreitetes Kommandozeilenprogramm in Linux. Seine Hauptaufgabe findet der Befehl im Archivieren und in der Kompression von Dateien. Ursprünglich wurde `tar` entwickelt, um Dateien auf Magnetbändern zu speichern. Heute wird `tar` jedoch auch für andere Speichermedien verwendet. Der Befehl `tar` kann somit auch für die Datei- und Verzeichnissicherung problemlos eingesetzt werden.

Grundlegende Syntax des Befehls

```
tar [options] [archive-file] [file or directory to be archived]
```

TERMINAL

- **[options] :**
Hier werden die unterschiedlichsten Optionen angegeben, die das Verhalten des Befehls modifizieren. So können beispielsweise Aktionen wie das Erstellen, Entpacken oder das Auflisten des Inhalts eines Archivs spezifiziert werden. Weitere gängige Optionen sind die Kompression und die Ausgabe im Verbose-Modus, welcher die Dateien während des Prozesses auflistet.
- **[archive-file] :**
Der Name der resultierenden Archivdatei, in der die angegebenen Dateien und Verzeichnisse gespeichert werden. Je nach verwendeter Kompression kann die Archivdatei die Endungen `.tar`, `.tar.gz` oder `.tar.bz2` aufweisen.
- **[file or directory to be archived] :**
Die Dateien oder Verzeichnisse, die im Archiv enthalten sein sollen. Es können mehrere Dateinamen und Verzeichnisse angegeben werden, die in einer einzigen Archivdatei zusammengefasst werden.

Die gängigsten Optionen des Befehls `tar` sind:

- **Create (-c) :**
Erstellt ein neues Archiv.
- **Extract (-x) :**
Extrahiert Dateien aus einem Archiv.
- **Verbose (-v) :**
Zeigt die detaillierte Ausgabe des Prozesses an.
- **File (-f) :**
Spezifiziert die Archivdatei.
- **Compress with gzip (-z) :**
Das Archiv wird mit gzip komprimiert, wodurch eine `.tar.gz` Datei entsteht.
- **Compress with bzip2 (-j) :**
Das Archiv wird mit bzip2 komprimiert, wodurch eine `.tar.bz2` Datei entsteht.
- **Compress with xz (-J) :**
Das Archiv wird mit xz komprimiert, wodurch eine `.tar.xz` Datei entsteht.

Durchführung von Vollsicherungen

TERMINAL

```
#!/bin/bash

if [ $# -ne 2 ]; then
    echo "Usage: $0 SOURCE BACKUP_DIR"
    exit 1
fi

DATE=$(date +%Y-%m-%d-%H%M%S) (1)
BACKUP_DIR="$2" (2)
SOURCE="$1" (3)

mkdir -p "$BACKUP_DIR" (4)

echo "Make backup $DATE"
echo "===== "

tar -cvzpf "$BACKUP_DIR/backup-$DATE.tar.gz" "$SOURCE" (5)

echo "===== "
echo "Backup finished"
```

- **(1) :**
Die Variable `DATE` wird mit dem aktuellen Datum und der aktuellen Uhrzeit befüllt. Dieses Datum wird später im Dateinamen des Backups verwendet. Das Format für den Timestamp wird dabei immer nach folgendem Muster aufgebaut: `Jahr-Monat-Tag-StundeMinuteSekunde`.
- **(2) :**
Der Pfad zum Quellverzeichnis, das gesichert werden soll, wird in der Variable `SOURCE` festgelegt.
- **(3) :**
Der Pfad zum Zielverzeichnis, in dem die Backups gespeichert werden sollen, wird in der Variable `BACKUP_DIR` festgelegt.
- **(4) :**
Das Zielverzeichnis für die Backups wird erstellt, falls es noch nicht existiert. Hierdurch wird sichergestellt, dass der Prozess ohne Fehler fortgesetzt werden kann, auch wenn das Zielverzeichnis zuvor nicht vorhanden war.

- (5) :

Der Befehl `tar` wird mit den Optionen `-cvzpf` ausgeführt. Dabei steht `-c` für das Erstellen eines neuen Archivs, `-v` für den Verbose-Modus, `-z` für die Kompression mit `gzip`, `-p` für die Beibehaltung der Dateirechte und `-f` für die Angabe des Dateinamens. Das Archiv wird im Zielverzeichnis mit dem Namen `backup-$(DATE).tar.gz` gespeichert.

Wiederherstellung einer Vollsicherung

Da bei einer Vollsicherung immer nur eine einzige Datei vorhanden ist, ist folglich die Wiederherstellung ebenfalls leichter:

```
#!/bin/bash
```

TERMINAL

```
DATE=$(date +%Y-%m-%d-%H%M%S) (1)
```

```
tar -xzf backup-$(DATE).tar.gz -C / (2)
```

- (1) :

Das Datum der entsprechenden Vollsicherung. Hier muss wieder das entsprechende Format eingehalten werden z. B. Jahr-Monat-Tag-StundeMinuteSekunde .

- (2) :

Der Befehl `tar` wird mit den Optionen `-xzf` ausgeführt. Dabei steht `-x` für das Extrahieren eines Archivs, `-z` für die Dekompression mit `gzip`, `-f` für die Angabe des Dateinamens und `-C` für das Verzeichnis, in das die Dateien extrahiert werden sollen.

Durchführung von inkrementellen Sicherungen

```
#!/bin/bash
```

TERMINAL

```
SOURCE="$1" (1)
```

```
DESTINATION="$2" (2)
```

```
SNAPSHOT="$DESTINATION/timestamp.dat" (3)
```

```
DATE=$(date +%Y-%m-%d-%H%M%S) (4)
```

```
mkdir -p "$DESTINATION" (5)
```

```
echo "Starting backup..."
```

```
tar -czpf "$DESTINATION/backup-$(DATE).tar.gz" -g "$SNAPSHOT" "$SOURCE" (6)
```

```
echo "Backup created: $DESTINATION/backup-$(DATE).tar.gz"
```

```
echo "=====
```

- (1) :

Der Pfad zum Quellverzeichnis, das gesichert werden soll, wird in der Variable `SOURCE` festgelegt.

- (2) :

Der Pfad zum Zielverzeichnis, in dem die Backups gespeichert werden sollen, wird in der Variable `DESTINATION` festgelegt.

- (3) :

Der Pfad zur Snapshot-Datei, die für die inkrementelle Sicherung verwendet wird. Diese Datei speichert Informationen über die Dateien, die in der vorherigen Sicherung enthalten waren, und ermöglicht es `tar`, nur die Dateien zu sichern, die seit der letzten Sicherung geändert wurden. Diese Datei ist folglich essentiell und darf nicht einfach geändert oder gar gelöscht werden, da sonst die Konsistenz der Sicherungen nicht mehr gewährleistet werden kann.

- (4) :
Die Variable `DATE` wird mit dem aktuellen Datum und der aktuellen Uhrzeit befüllt. Dieses Datum wird später im Dateinamen des Backups verwendet. Das Format für den Timestamp wird dabei immer nach folgendem Muster aufgebaut: `Jahr-Monat-Tag-StundeMinuteSekunde`.
- (5) :
Das Zielverzeichnis für die Backups wird erstellt, falls es noch nicht existiert. Hierdurch wird sichergestellt, dass der Prozess ohne Fehler fortgesetzt werden kann, auch wenn das Zielverzeichnis zuvor nicht vorhanden war.
- (6) :
Der Befehl `tar` wird mit den Optionen `-cpzf` ausgeführt. Dabei steht `-c` für das Erstellen eines neuen Archivs, `-p` für die Beibehaltung der Dateirechte, `-z` für die Kompression mit `gzip`, `-f` für die Angabe des Dateinamens, `-g` für die Angabe der Datei, in der der Zeitstempel gespeichert wird. Das Archiv wird im Zielverzeichnis mit dem Namen `backup- $\{DATE\}$ .tar.gz` gespeichert.

Wiederherstellung einer inkrementellen Sicherung

Bei der Wiederherstellung von inkrementellen Backups sind immer mehrere Dateien betroffen, daher muss sichergestellt werden, dass auch wirklich alle Dateien im Zugriff sind.

```
#!/bin/bash
```

TERMINAL

```
BACKUP_DIR="$1" (1)
```

```
for archiv in "${BACKUP_DIR}"/backup-*.tar.gz; do (2)
```

```
    tar -xpf "$archiv" -C / (3)
```

```
done
```

- (1) :
Der Pfad zum Zielverzeichnis, in dem die Backups gespeichert sind, wird in der Variable `BACKUP_DIR` festgelegt.
- (2) :
Es wird eine Schleife über alle Dateien im Backup-Verzeichnis durchgeführt.
- (3) :
Der Befehl `tar` wird mit den Optionen `-xpf` ausgeführt. Dabei steht `-x` für das Extrahieren eines Archivs, `-p` für die Beibehaltung der Dateirechte, `-z` für die Dekompression mit `gzip`, `-f` für die Angabe des Dateinamens und `-C` für das Verzeichnis, in das die Dateien extrahiert werden sollen. In diesem Fall wird das Archiv in das Wurzelverzeichnis `/` extrahiert.

Da die Dateien für die Backups durchnummeriert bzw. mit einem Timestamp vorbelegt sind, werden alle Dateien im Backup-Verzeichnis nacheinander extrahiert. Dabei wird darauf geachtet, dass die Dateien in der richtigen Reihenfolge extrahiert werden, um die Konsistenz der Daten zu gewährleisten. Man beginnt mit dem ältesten und arbeitet sich durch bis zum neuesten Backup.

Weitere Sicherungsarten

Sicherungen via SSH

Eine gute Möglichkeit für entsprechende Backups ist auch die Sicherung und Speicherung der Backups auf einem entfernten Rechner. Hierfür kann ebenfalls `tar` verwendet werden. Der Vorteil hierbei ist, dass die Backups auf einem anderen Rechner gespeichert werden und somit auch bei einem Totalausfall des Systems noch vorhanden sind. Die Sicherung via SSH kann wie folgt aussehen:

```
tar czvf - xy | ssh ux4 "cat > Sicherung/xy.tgz"
```

TERMINAL

Im aufgeführten Beispiel wird das Verzeichnis `xy` auf dem lokalen Rechner archiviert und anschließend via SSH auf den Rechner `ux4` übertragen. Dort wird die Datei `xy.tgz` im Verzeichnis `Sicherung` abgelegt. Relevant ist das `-` Zeichen. Nach dem Parameter `-f` bei `tar` folgt immer ein Dateiname. Wird hier das `-` Zeichen verwendet, wird die Ausgabe von `tar` an den Standard-Output weitergeleitet. Dieser wird dann von `ssh` genutzt, um die Datei auf den entfernten Rechner zu übertragen. `-` steht für `STDIN` bzw. `STOUT`.

Um die entsprechende Sicherung wiederherzustellen, wird nachfolgender Befehl genutzt:

```
ssh ux4 "cat Sicherung/xy.tgz" | tar xzvf -
```

TERMINAL

Erstellung von Snapshots

Snapshots sind mehr oder weniger Abbilder eines entsprechenden Systems zu einem bestimmten Zeitpunkt. Das jeweilige System wird hierfür "eingefroren", was bedeutet, dass ab diesem Moment sich die Daten nicht mehr verändern können. Der Snapshot kann im Anschluss als Basis für eine Sicherung verwendet werden. Ein einfaches Beispiel hierfür könnte folgendermaßen aussehen: (vgl. [SNA1])

Man hat auf einem System eine grundlegende Datenmenge von 20 TB. Die komplette Datenmenge soll gesichert werden. Zur Sicherung steht ein Laufwerk mit einer Übertragungsrate von 160 MB/s zur Verfügung.

Die ersten Snapshot Technologien sind für schnelle Datensicherung und Rücksicherungen mit geringer Downtime entwickelt worden. Müssen Datenmengen von mehreren Terabyte gesichert werden, so dauert dies relativ lange. Ein einfaches Beispiel: Sie möchten 20 TB mit einem LTO-6 Laufwerk sichern. Dieses Laufwerk schafft eine Transferrate von 160 MB/s (unkomprimiert). Sie benötigen also ca. 36 Stunden für die Sicherung. Jetzt brauchen wir natürlich eine konsistente Sicherung, aber können Applikationen und Datenbanken meist nur für eine sehr kurz Zeit heruntergefahren. Also kurzer Stopp für die Erstellung des Snapshots (Zustandskopie nicht Datenkopie) und dann kann von der Kopie (dem Snapshot) gesichert werden, während die Applikation oder Datenbank schon wieder läuft. Die Vollsicherung dauert immer noch 36 Stunden, aber die Applikation oder Datenbank kann nach wenigen Sekunden wieder gestartet werden.

Anmerkung: Wenn die Datenbank oder Anwendung einen Backup-Modus besitzt, muss die Applikation noch nicht einmal kurz gestoppt werden.

Sicherung von Datenbanken

Natürlich sind auch Datenbanken wie MySQL, MariaDB oder PostgreSQL entsprechend zu sichern. Bereits ein Ausfall, ein fehlerhaftes Update oder eine fehlerhafte Konfiguration kann zum Kompletterverlust der Daten führen. Im nachfolgenden Teilkapitel wird die Funktionsweise eines Backups innerhalb einer MariaDB Datenbank demonstriert. (vgl. [BAC9])

Logische vs. physische Sicherung

Es gibt grundsätzlich zwei unterschiedliche Arten wie eine Datenbanksicherung durchgeführt werden kann: (vgl. [BAC9])

- Logische Sicherung:

Bei der logischen Sicherung werden die Daten in einem für Menschen lesbaren Format exportiert. Hierfür werden wirklich alle relevanten Anweisungen genutzt, damit die komplette Datenbankstruktur und sämtliche Daten wiederhergestellt werden können. Diese Art von Sicherung wird hauptsächlich für den Umzug auf andere Systeme. (vgl. [BAC9])

- Physische Sicherung:

Bei der physischen Sicherung werden die eigentlichen Datenbankdateien kopiert. Diese Art der Sicherung ist wesentlich schneller als die logische Sicherung, da hier keine Export- und Importvorgänge stattfinden müssen. Allerdings ist diese Art der Sicherung nur auf dem gleichen Datenbanksystem wiederherstellbar. (vgl. [BAC9])

Beispiel: Logische Sicherung mit `mariadb-dump`

MariaDB hat ein sehr bekanntes Tool namens `mariadb-dump`, welches für die logische Sicherung von Datenbanken genutzt werden kann. Mit diesem Tool können einzelne Datenbanken, aber auch komplette MariaDB Server gesichert werden. (vgl. [BAC9])

Zu den wesentlichen Vorteilen von `mariadb-dump` zählen: (vgl. [BAC9])

- Einsatzfähigkeit auf jedem System mit einem MariaDB Client.
- Kann Tabellen und Datenbanken selektiv sichern.
- Ist zu anderen MariaDB bzw. MySQL Versionen kompatibel.

Nachteile von `mariadb-dump` sind: (vgl. [BAC9])

- Die Sicherung kann bei sehr großen Datenbanken relativ lange dauern.
- Sicherung kann zu Lastspitzen führen.

Sicherung aller Datenbanken

Um alle Datenbanken auf einem MariaDB Server zu sichern, kann folgender Befehl genutzt werden:

```
mariadb-dump -u root -p --all-databases --result-file=/backups/backup.sql
```

TERMINAL

- `-u root` : Der Benutzername für die Verbindung zur Datenbank.
- `-p` : Fordert zur Eingabe des Passworts auf.
- `--all-databases` : Sichert alle Datenbanken auf dem Server.
- `--result-file=/backups/backup.sql` : Gibt die Datei an, in der die Sicherung gespeichert wird.

Bestimmte Datenbank sichern

Um eine bestimmte Datenbank zu sichern, kann folgender Befehl genutzt werden:

```
mariadb-dump -u root -p --databases <NameOfDatabase> --result-file=/backups/backup.sql
```

TERMINAL

Durch den Befehl werden auch sämtliche `CREATE DATABASE` Anweisungen in die Sicherung aufgenommen. Möchte man dies vermeiden, so wieder der Befehl einfach nur leicht abgewandelt:

```
mariadb-dump -u root -p <NameOfDatabase> --result-file=/backups/backup.sql
```

TERMINAL

Mit Hilfe des des Zusatzes `--databases` können auch mehrere Datenbanken gleichzeitig zur Sicherung hinzugefügt werden. Die Datenbanken werden dabei einfach nacheinander angegeben.

Einzelne Tabelle(n) sichern

Um nur bestimmte Tabellen einer Datenbank zu sichern, kann folgender Befehl genutzt werden:

```
mariadb-dump -u root -p <NameOfDatabase> <Table1> <Table2> --result-file=/backups/backup.sql
```

TERMINAL

Hierbei werden nur die angegebenen Tabellen `Table1` und `Table2` aus der Datenbank `NameOfDatabase` gesichert.